

DSC602 Information Visualization - June 2025 Exam Solutions

University of Buea | Faculty of Science | 70 Total Marks

Exam Details:

- Date: Saturday 28/06/2025
 - Time: 08:00 - 11:00 AM (3 hours)
 - Course: DSC602 - Information Visualization
 - Instructor: Dr. Nyamsi
 - Total Marks: 70
 - Credit Value: 6
-

QUESTION 1: READING AND COMPREHENSION (25 marks)

Q1.1: What is Meant by Information Visualization? (6 marks - 3+3)

PART A: Definition (3 marks)

Information Visualization (InfoVis) Definition:

Information Visualization is the discipline of representing abstract, non-spatial data in visual and interactive forms to support human perception, cognition, and decision-making. It transforms data that lacks inherent spatial dimensions into visual representations that humans can readily interpret.

Key Components:

1. **Input:** Abstract data with no inherent 2D/3D structure
 - Examples: Financial data, text documents, relationships, hierarchies
 - NOT naturally spatial (unlike maps or scientific data)
2. **Process:** Transformation through visual encoding
 - Map data attributes to visual properties (color, size, position, shape)
 - Create perceptually effective representations
 - Enable interactive exploration
3. **Output:** Visual/interactive representations
 - Enables pattern recognition
 - Supports analytical reasoning
 - Facilitates insight generation
 - Communicates findings effectively

Formal Definition (from Lectures):

"The use of computer-supported, interactive, visual representations of abstract data to amplify cognition and

enable new discoveries."

Why Separate from General Visualization:

- **Scope:** Focuses on abstract information (not naturally spatial)
 - **Goal:** Enable analytical discovery (not just display)
 - **Interaction:** Heavy emphasis on interactivity and exploration
 - **Complexity:** Handles highly multi-dimensional data
-

PART B: Its Meaning in Context of Modern Data (3 marks)

Meaning in Modern Context:

In today's data-rich world, Information Visualization means:

1. Necessity for Understanding

- Organizations generate terabytes of data daily
- No human can read raw numbers effectively
- Visual representation becomes ESSENTIAL for comprehension
- Makes invisible patterns VISIBLE

2. Enabling Decision-Making

- Executives need insights, not data
- Visualizations communicate complex patterns instantly
- Supports data-driven decision making
- Reduces time from data to actionable insight

3. Handling Complexity

- Modern data is multi-dimensional (10+ variables)
- Traditional tables cannot show all dimensions
- Visualization handles complexity through clever encoding
- Coordinated views enable holistic understanding

4. Democratizing Analytics

- Non-technical users can explore data
- Interactive dashboards enable self-service analytics
- Business users don't need data science background
- Empowers entire organization

Example Application:

- **Raw Data:** 1 million sales records with 20 attributes
 - **Table View:** Incomprehensible; impossible to spot patterns
 - **Visualization:** Instant insight (top products, regional trends, seasonal patterns)
 - **Meaning:** Transforms overwhelming data into actionable intelligence
-

Q1.2: Why Do Modern Businesses Need Data Visualization? (6 marks)

ANSWER FRAMEWORK: 6 marks = 6 key reasons, 1 mark each

Reason 1: Cognitive Advantage (1 mark)

Fact: Humans process visual information 60,000x faster than text.

- A table of numbers takes minutes to understand
- Same data as a chart is comprehensible in seconds
- Reduces cognitive load
- Example: Sales dashboard shows quarterly trends instantly

Impact: Faster insights = quicker decision-making = competitive advantage

Reason 2: Pattern Recognition (1 mark)

Mechanism: Human vision evolved to detect patterns in complex scenes.

- Visual cortex excels at:
 - Identifying outliers immediately
 - Spotting trends and clusters
 - Recognizing correlations
 - Finding anomalies
- Raw data hides these patterns
- Visualization makes them obvious

Business Impact: Discover problems before they become crises

- Detect fraudulent transactions instantly
- Identify underperforming products
- Spot supply chain disruptions

Reason 3: Decision Support (1 mark)

Context: Modern businesses make decisions based on data, not intuition.

- Wrong decisions cost money (missed opportunities, inefficiency)
- Data-driven decisions are more accurate
- Visualizations support both:
 - **Strategic decisions:** Long-term planning with trend analysis
 - **Tactical decisions:** Day-to-day optimization
 - **Operational decisions:** Real-time adjustments

Examples:

- Inventory optimization (which products to stock?)
- Marketing budget allocation (which channels work best?)
- Resource scheduling (when to hire?)

Reason 4: Competitive Advantage (1 mark)

Market Reality: Companies with better data insights win.

- Netflix: Visualization of viewer behavior → recommendations
- Amazon: Purchase pattern visualization → personalization
- Healthcare: Patient data visualization → better diagnoses
- Finance: Risk visualization → better portfolio management

Value:

- Companies using data visualization show 50%+ higher ROI
- Data-driven competitors outpace intuition-based ones
- First-mover advantage in visualization adoption

Reason 5: Communication and Storytelling (1 mark)

Challenge: Communicating insights across organization

- Executives don't read 50-page reports
- Non-technical stakeholders don't understand statistics
- Traditional presentations are boring and forgettable

Solution: Visualizations communicate complex ideas simply

- One chart replaces 10 pages of text
- Visual stories are memorable
- Dashboards enable self-service exploration
- Stakeholders understand and trust visual evidence

Business Context:

- Board presentations (convince leadership)
- Client presentations (demonstrate value)
- Team communication (align departments)
- Public reporting (build confidence)

Reason 6: Handling Big Data Scale (1 mark)

Problem: Modern datasets are massive

- Traditional analysis methods overwhelmed by volume
- Cannot examine every data point manually
- Excel sheets crash with millions of rows
- Summary statistics hide important details

Solution: Visualization techniques that scale

- Aggregation: Show summaries without losing detail
- Interaction: Users drill down to details as needed
- Progressive disclosure: Start simple, get complex
- Linked views: Multiple perspectives on same data

Example: Monitoring 1 million sensors

- Cannot read 1 million numbers
 - Visualization shows status instantly (red = problem, green = normal)
 - Drill-down capability for investigation
-

Q1.3: How Can Modern Governance Exploit Information Visualization to Make Decisions? (6 marks)

Context: Governance = government, public administration, policy-making

GOVERNANCE CHALLENGES

- Complex societal problems (education, health, economy, environment)
- Multiple stakeholders with conflicting interests
- Need to balance fiscal responsibility with social needs
- Limited resources requiring optimization
- Public accountability and transparency

HOW VISUALIZATION HELPS (6 marks = 6 applications, 1 mark each)

Application 1: Performance Monitoring (1 mark)

Use Case: Track government agency performance

- **Healthcare:** Visualize hospital performance (patient satisfaction, wait times, mortality rates)
- **Education:** Visualize school performance (graduation rates, test scores, enrollment)
- **Finance:** Visualize tax collection vs. spending

Visualization Type: Dashboards with KPIs

- Color-coded status (red/yellow/green)
- Trend lines showing improvement/decline
- Comparison with targets and peers

Decision Impact: Identify underperforming services requiring intervention

Application 2: Budget Allocation (1 mark)

Use Case: How should government allocate limited budgets?

- Example: Municipal government with 100M budget across 50 departments
- Which departments need more funding?
- Are resources aligned with population needs?

Visualization Technique:

- Treemap: Show budget allocation by size and category
- Breakdown: Department → Sub-department → Line items
- Comparison: Budget vs. actual spending vs. outcomes

Decision Impact: Optimize resource allocation for maximum impact

Application 3: Policy Impact Assessment (1 mark)

Use Case: Evaluate if policies are working

- Example: "New traffic regulation reduced accidents by X%?"
- "Subsidy policy for farmers increased crop yield?"
- "Tax incentive for investment attracted businesses?"

Visualization:

- Before/After comparisons
- Time series showing trend
- Geographic maps showing regional effects
- Demographic breakdowns

Decision Impact: Determine if policy should continue, modify, or discontinue

Application 4: Citizen Engagement and Transparency (1 mark)

Use Case: Communicate with citizens about governance

- Public budget transparency (where does tax money go?)
- Service access (hospitals, schools, police stations near me?)
- Environmental monitoring (air quality, water quality in my area?)
- Election results and voting patterns

Visualization Style:

- Interactive web-based dashboards
- Simple, jargon-free visualizations
- Geographic maps for spatial data
- Accessible design for diverse audiences

Decision Impact: Build public trust; enable informed citizens to participate in democracy

Application 5: Identifying Social Issues (1 mark)

Use Case: Discover problems requiring policy intervention

- Where is poverty concentrated?
- Which regions have highest crime rates?
- Where are disease outbreaks occurring?
- Which populations have lowest access to services?

Visualization Types:

- Heat maps: Identify hotspots
- Scatter plots: Show correlations (poverty ↔ education)
- Time series: Track disease spread

- Network maps: Show service accessibility

Decision Impact: Prioritize interventions where they'll have most impact

Application 6: Comparative Analysis and Benchmarking (1 mark)

Use Case: How does our region compare to others?

- How does country's GDP growth compare to regional peers?
- How do our schools perform vs. national averages?
- How is our healthcare system compared to developed nations?

Visualization:

- Bar charts: Direct comparisons
- Parallel coordinates: Compare multiple metrics simultaneously
- Ranked lists: Show position in rankings
- Spider diagrams: Multi-dimensional comparison

Decision Impact: Identify best practices; set realistic benchmarks; adopt successful policies

Real-World Examples:

1. **Singapore:** Uses dashboards to monitor smart city initiatives
2. **Denmark:** Visualizes government spending for transparency
3. **USA:** CDC dashboards for disease tracking and response
4. **UN:** Sustainable Development Goals tracked with visualizations

Q1.4: Describe 3 Different Graphics Projected in Class and Explain Why They Are Useful (6 marks)

ANSWER STRUCTURE: 3 graphs × 2 marks each = 6 marks

- 1 mark: Describe the graph (how it looks, what it shows)
- 1 mark: Explain why it's useful (specific advantages)

GRAPH 1: LINE GRAPH (2 marks)

Description (1 mark):

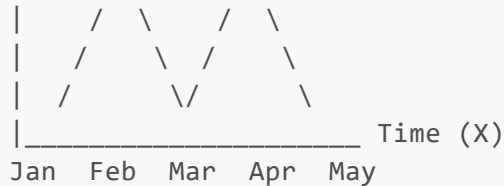
A line graph displays data points connected by lines, showing change over time or ordered categories.

- **X-axis:** Time (years, months, days) or ordered categories
- **Y-axis:** Quantitative values (sales, temperature, population)
- **Marks:** Points connected by lines
- **Channels:** Position (x,y), color (multiple series), line style (trend types)

Visual Example:

Sales (Y)





Why Useful (1 mark):

- **Temporal Trends:** Instantly shows if data is increasing, decreasing, or stable
- **Pattern Recognition:** Reveals cycles, seasonality, and anomalies
- **Prediction:** Observers can extrapolate future trends
- **Multiple Series:** Can compare different time series on same graph (e.g., sales vs. costs)
- **Intuitive:** Most familiar to general audiences
- **Efficient:** Shows 2 dimensions clearly on standard 2D plot

Business Examples:

- Stock prices over time (detect market trends)
- Website traffic over months (see seasonality)
- Temperature trends (climate monitoring)
- Patient recovery progress (healthcare)

GRAPH 2: BAR CHART (2 marks)

Description (1 mark):

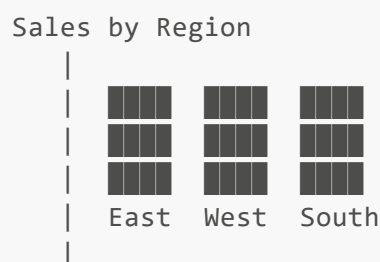
A bar chart displays values as rectangular bars, with height (or length) representing magnitude.

- **X-axis:** Categories or groups
- **Y-axis:** Quantitative values
- **Marks:** Rectangles (bars)
- **Channels:** Position (category), Length/Height (magnitude), Color (optional grouping)

Variants:

- Horizontal bars (better for long category names)
- Grouped bars (compare multiple values within each category)
- Stacked bars (show composition: part-to-whole relationships)

Visual Example:



Why Useful (1 mark):

- **Precise Comparison:** Bar heights directly comparable (accurate magnitude perception)
- **Multiple Categories:** Can show many categories without confusion
- **Composition:** Stacked variant shows part-to-whole (e.g., market share by company)
- **Simplicity:** Easy for general audience to understand
- **Flexibility:** Works with categorical, ordinal, or continuous data
- **Value Reading:** Can add labels for exact values

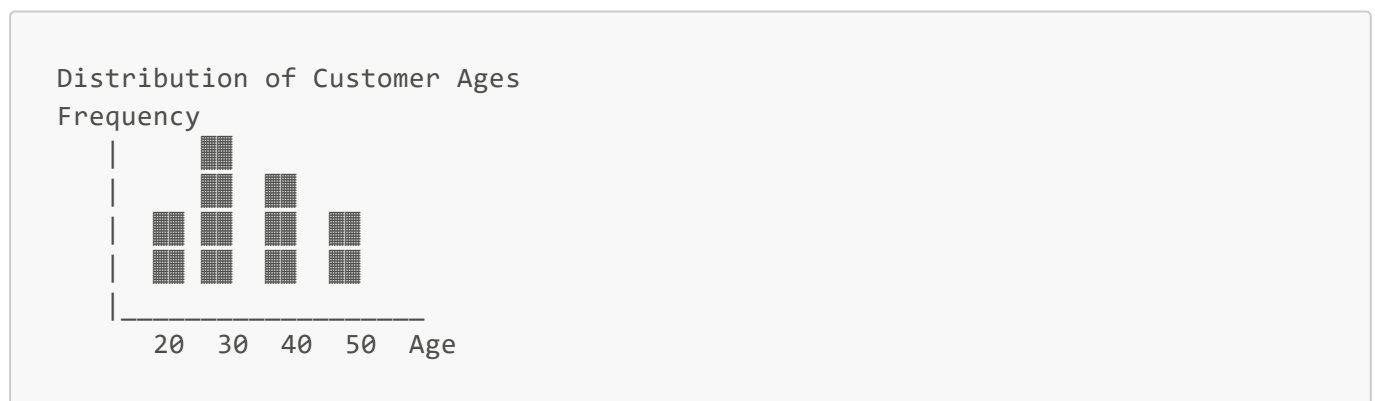
Business Examples:

- Sales by product (which products generate revenue?)
- Performance by department (which department underperforms?)
- Market share by competitor (competitive positioning)
- Survey responses (how many chose option A vs. B?)
- Gender distribution (demographic breakdown)

GRAPH 3: HISTOGRAM (2 marks)**Description** (1 mark):

A histogram shows the distribution of a continuous variable, with bars representing frequency in ranges (bins).

- **X-axis:** Continuous variable ranges (bins: 0-10, 10-20, 20-30)
- **Y-axis:** Frequency (count or percentage)
- **Marks:** Rectangular bars (touching, unlike bar chart)
- **Channels:** Position (range), Height (frequency)

Visual Example:**Why Useful** (1 mark):

- **Distribution Shape:** Instantly reveal if data is normal, skewed, bimodal, etc.
- **Outlier Detection:** Identify unusual values outside typical range
- **Central Tendency:** See where most data concentrates
- **Spread:** Understand data variability and range
- **Quality Control:** Detect manufacturing defects (out-of-specification items)
- **Statistical Insights:** Foundations for further analysis

Business Examples:

- Customer age distribution (who are our customers?)
- Income distribution (wealth inequality analysis)
- Product defect rates (quality assessment)
- Response time distribution (service performance)
- Test score distribution (education assessment)

Why These 3 Graphs Are Essential:

Graph	Best For	Key Advantage
Line	Trends over time	Shows temporal patterns instantly
Bar	Comparing categories	Precise, easy to read
Histogram	Distribution analysis	Reveals data shape and outliers

Together, these three cover ~70% of business visualization needs:

- Track performance over time (Line)
- Compare current performance (Bar)
- Understand data characteristics (Histogram)

QUESTION 2: DATA DESCRIPTION AND PREPARATION (20 marks)

Q2.1: Describe Dictionary, Tuple, and Series (9 marks - 3×3)

Context: Python data structures used in data visualization pipelines

DATA STRUCTURE 1: DICTIONARY (3 marks)

Definition (1 mark):

A Dictionary is an unordered collection of key-value pairs enclosed in curly braces `{}`. Each unique key maps to exactly one value.

Syntax:

```
# Basic dictionary
person = {'name': 'John', 'age': 30, 'city': 'London', 'salary': 50000}

# Dictionary with mixed types
data = {'employees': 100, 'revenue': 1000000.50, 'status': True, 'departments':
['IT', 'HR', 'Sales']}
```

Key Characteristics (1 mark):

- **Unordered:** No guarantee of element order (Python 3.7+ maintains insertion order, but not logically important)

- **Mutable:** Can be modified (add, remove, update items)
- **Key-Value Pairs:** Each element is a unique key linked to a value
- **Flexible Types:** Keys and values can be different types (though keys usually strings/integers)
- **Access by Key:** Retrieve values using `dictionary[key]`, not by position

Data Visualization Application (1 mark):

Metadata and Configuration:

```
# Configuration for visualization
config = {
    'plot_type': 'bar',
    'colors': ['red', 'blue', 'green'],
    'title': 'Sales by Region',
    'xlabel': 'Region',
    'ylabel': 'Sales (Million $)',
    'figure_size': (12, 6)
}

# Data with labels
country_data = {
    'France': 85,
    'Germany': 95,
    'Italy': 62,
    'Spain': 48,
    'Portugal': 32
}

# Visualization loop
for country, gdp in country_data.items():
    print(f"{country}: {gdp}B EUR")
```

Why Important for Visualization:

- Store and pass visualization parameters
- Map category names to values
- Organize multi-dimensional data
- Flexible key-based access (more intuitive than numeric indices)

DATA STRUCTURE 2: TUPLE (3 marks)

Definition (1 mark):

A Tuple is an ordered, immutable collection of elements enclosed in parentheses `()`. Once created, it cannot be modified (no adding, removing, or changing elements).

Syntax:

```
# Basic tuple
coordinates = (x, y)
```

```
rgb_color = (255, 128, 0)
point_in_3d = (10.5, 20.3, 15.7)

# Unpacking tuple
x, y = coordinates
r, g, b = rgb_color
```

Key Characteristics (1 mark):

- **Ordered:** Elements have fixed positions (index 0, 1, 2, ...)
- **Immutable:** Cannot be changed after creation
- **Indexed Access:** Retrieve by position: `tuple[0]`, `tuple[1]`
- **Lightweight:** More memory efficient than lists
- **Hashable:** Can be used as dictionary keys (unlike lists)
- **Unpacking:** Can assign multiple values: `a, b, c = tuple_of_3`

Data Visualization Application (1 mark):**Coordinate Storage and Graphical Properties:**

```
# Store 2D/3D coordinates
points = [
    (10, 20),    # (x, y)
    (15, 25),
    (20, 30)
]

# Store colors as RGB tuples
colors = [
    (255, 0, 0),    # Red
    (0, 255, 0),    # Green
    (0, 0, 255)     # Blue
]

# Store dimension sizes
figure_size = (12, 8)    # (width, height)
position = (100, 150)    # (x_pixel, y_pixel)

# Return multiple values from function
def get_plot_range(data):
    return (min(data), max(data)) # Returns tuple

min_val, max_val = get_plot_range([1, 5, 3, 8, 2])
```

Why Important for Visualization:

- Store fixed coordinate pairs (efficient memory)
- Immutability ensures data integrity
- Represent colors (RGB values don't change)
- Return multiple values from functions

- Use as dictionary keys for unique visual properties

DATA STRUCTURE 3: SERIES (3 marks)

Definition (1 mark):

A Pandas Series is a 1-dimensional labeled array that contains data of any type and an associated axis labels (index). It's built on NumPy and designed for data manipulation and analysis.

Syntax:

```
import pandas as pd

# From list
ages = pd.Series([25, 30, 35, 40, 45], index=['Alice', 'Bob', 'Carol', 'David', 'Eve'])

# From dictionary
sales = pd.Series({
    'Q1': 100000,
    'Q2': 150000,
    'Q3': 120000,
    'Q4': 180000
})

# Access elements
sales['Q1']          # 100000
sales[0]             # 100000 (first element)
sales[['Q1', 'Q3']] # Multiple elements
```

Key Characteristics (1 mark):

- **Labeled:** Each value has an associated index label (not just numeric position)
- **Homogeneous:** All elements typically same data type (can mix, but not recommended)
- **Mutable:** Can be modified (add, remove, change values)
- **Operations:** Supports mathematical operations, filtering, aggregation
- **Integration:** Works seamlessly with DataFrames and visualization libraries
- **Efficient:** Optimized for data operations using NumPy under the hood

Data Visualization Application (1 mark):

Core Data for Visualization:

```
# Sales data
sales_series = pd.Series(
    [100000, 150000, 120000, 180000],
    index=['Q1', 'Q2', 'Q3', 'Q4'],
    name='Quarterly Sales'
)
```

```
# Visualization
import matplotlib.pyplot as plt
sales_series.plot(kind='bar', title='Sales by Quarter')
plt.ylabel('Sales ($)')
plt.show()

# Regional data
regional_sales = pd.Series({
    'North': 500000,
    'South': 350000,
    'East': 600000,
    'West': 450000
})

# Filtering for visualization
high_sales = regional_sales[regional_sales > 400000]
# Use high_sales for visualization

# Time series
import numpy as np
dates = pd.date_range('2025-01-01', periods=100)
values = np.random.randn(100).cumsum()
time_series = pd.Series(values, index=dates)
time_series.plot() # Automatic time axis formatting
```

Why Important for Visualization:

- Direct integration with matplotlib (`.plot()` method)
- Labeled indices provide semantic meaning
- Filtering and transformation before visualization
- Time series support with date indices
- Built-in statistical methods for exploratory analysis
- Efficient memory usage for large datasets

Q2.2: How Can Each Collection Be Obtained, Maintained, and Used for Data Visualization? (6 marks)

Answer Structure: 6 marks = 2 marks per structure (obtain + maintain + use)

DICTIONARY - OBTAIN, MAINTAIN, USE (2 marks)

OBTAIN (How to Create/Get Dictionaries) (0.5 marks):

1. Direct Creation:

```
students = {'Alice': 85, 'Bob': 92, 'Carol': 78}
```

2. From Lists of Pairs:

```
names = ['Alice', 'Bob', 'Carol']
scores = [85, 92, 78]
students = dict(zip(names, scores))
```

3. From JSON/File:

```
import json
with open('data.json') as f:
    data = json.load(f) # Loads as dictionary
```

4. From DataFrame:

```
df_dict = df.to_dict() # Convert DataFrame to dictionary
```

MAINTAIN (How to Update/Keep Consistent) (0.5 marks):

1. **Add/Update:** `students['David'] = 88`
2. **Delete:** `del students['Carol']`
3. **Merge:** `students.update({'Eve': 95, 'Frank': 80})`
4. **Validation:** Check if key exists: `if 'Alice' in students:`

USE FOR VISUALIZATION (1 mark):

```
import matplotlib.pyplot as plt

# Visualization configuration (dictionary)
config = {
    'figsize': (12, 6),
    'colors': ['#FF6B6B', '#4ECDC4', '#45B7D1'],
    'title': 'Student Scores',
    'ylabel': 'Score'
}

# Data (dictionary)
students = {'Alice': 85, 'Bob': 92, 'Carol': 78, 'David': 88}

# Create visualization
fig, ax = plt.subplots(figsize=config['figsize'])
bars = ax.bar(students.keys(), students.values(), color=config['colors'])
ax.set_title(config['title'])
ax.set_ylabel(config['ylabel'])
plt.show()
```

Value for Visualization: Stores configuration, maps data to labels, enables flexible parameterization.

TUPLE - OBTAIN, MAINTAIN, USE (2 marks)

OBTAIN (How to Create Tuples) (0.5 marks):

1. Direct Creation:

```
point = (10, 20)
rgb = (255, 128, 0)
```

2. From Lists:

```
coords = tuple([10, 20, 30])
```

3. Unpacking:

```
x, y = (10, 20)
```

4. From Function Returns:

```
def get_range(data):
    return (min(data), max(data))
```

MAINTAIN (Constraints - Immutable) (0.5 marks):

1. Immutable: Cannot change after creation

```
point = (10, 20)
# point[0] = 15 ← Error! Cannot modify
```

2. Replace Entirely:

```
point = (15, 25) # Create new tuple, don't modify
```

3. Consistency: Good for fixed coordinates/colors

USE FOR VISUALIZATION (1 mark):


```
import matplotlib.pyplot as plt
import matplotlib.patches as patches

# Color definitions (tuples - immutable)
colors = {
    'red': (1.0, 0.0, 0.0),
    'green': (0.0, 1.0, 0.0),
    'blue': (0.0, 0.0, 1.0)
}

# Coordinate storage (tuples)
points = [(10, 20), (15, 25), (20, 30), (25, 35)]

# Create scatter plot
fig, ax = plt.subplots()
x_vals = [p[0] for p in points] # Extract x coordinates
y_vals = [p[1] for p in points] # Extract y coordinates
ax.scatter(x_vals, y_vals, color=colors['red'], s=100)
ax.set_title('Scatter Plot with Tuple Coordinates')
plt.show()

# Figure size specification
fig_size = (12, 8)
fig, ax = plt.subplots(figsize=fig_size)
```

Value for Visualization: Efficient storage of coordinates, fixed color specifications, reliable function return values.

SERIES - OBTAIN, MAINTAIN, USE (2 marks)

OBTAIN (How to Create Series) (0.5 marks):

1. Direct Creation:

```
import pandas as pd
sales = pd.Series([100, 150, 120], index=['Q1', 'Q2', 'Q3'])
```

2. From Dictionary:

```
data = {'Q1': 100, 'Q2': 150, 'Q3': 120}
sales = pd.Series(data)
```

3. From CSV/Excel:

```
df = pd.read_csv('sales.csv')
sales_column = df['Sales'] # Extracts as Series
```

4. Calculations:

```
sales = pd.Series([100, 150, 120])
growth = sales.pct_change() # New Series of percentage changes
```

MAINTAIN (Update and Transform) (0.5 marks):

1. Modify Values:

```
sales['Q1'] = 110 # Update specific value
sales[0] = 110    # Update by position
```

2. Add Missing Values Handling:

```
sales = sales.fillna(0) # Fill NaN with 0
sales = sales.dropna()  # Remove NaN values
```

3. Reindex:

```
sales = sales.reindex(['Q1', 'Q2', 'Q3', 'Q4']) # Add Q4 with NaN
```

USE FOR VISUALIZATION (1 mark):

```
import matplotlib.pyplot as plt
import pandas as pd

# Create Series
quarterly_sales = pd.Series(
    [100000, 150000, 120000, 180000],
    index=['Q1', 'Q2', 'Q3', 'Q4'],
    name='Sales'
)

# Multiple visualization options
fig, axes = plt.subplots(2, 2, figsize=(12, 8))

# Line plot
quarterly_sales.plot(ax=axes[0, 0], kind='line', title='Trend')
```

```
# Bar plot
quarterly_sales.plot(ax=axes[0, 1], kind='bar', title='Comparison')

# Histogram
quarterly_sales.plot(ax=axes[1, 0], kind='hist', title='Distribution')

# Area plot
quarterly_sales.plot(ax=axes[1, 1], kind='area', title='Cumulative')

plt.tight_layout()
plt.show()

# Filtering before visualization
high_sales = quarterly_sales[quarterly_sales > 120000]
high_sales.plot(kind='bar', title='High Sales Quarters')
```

Value for Visualization: Direct plotting capability, integrated statistical operations, efficient label handling, ideal for time series.

Q2.3: Data Frame Particularity for Efficient Programming (5 marks - 3+2)

What Is a Data Frame? (3 marks)

Definition (1 mark):

A DataFrame is a 2-dimensional, labeled data structure with rows and columns, where each column is a Series (1D array of data). It's the primary data structure in pandas for data manipulation and analysis.

Structure Visualization:

DataFrame Structure:

	Name	Age	Salary	Department
0	Alice	30	50000	IT
1	Bob	25	45000	HR
2	Carol	35	60000	IT
3	David	28	52000	Sales

Rows (Index): 0, 1, 2, 3 (labels each row)

Columns: Name, Age, Salary, Department (labeled each column)

Data Types: object, int64, int64, object

Key Characteristics (1 mark):

- **2D Structure:** Rows × Columns organization
- **Labeled:** Both row indices and column names (semantic meaning)
- **Heterogeneous:** Different columns can have different data types
- **Mutable:** Can add/remove/modify rows and columns
- **Indexed:** Supports both label-based and position-based access
- **Operations:** Built-in methods for filtering, grouping, aggregation, transformation

Syntax:

```
import pandas as pd

# Create DataFrame from dictionary
data = {
    'Name': ['Alice', 'Bob', 'Carol', 'David'],
    'Age': [30, 25, 35, 28],
    'Salary': [50000, 45000, 60000, 52000],
    'Department': ['IT', 'HR', 'IT', 'Sales']
}
df = pd.DataFrame(data)

# Access data
df['Name']          # Column as Series
df.loc[0]           # Row 0 as Series
df.iloc[0, 0]       # Element at row 0, col 0
df[df['Age'] > 28]   # Filter rows
```

Why DataFrames Are Particularly Efficient for Programming (2 marks)**Efficiency Aspect 1: Vectorized Operations** (1 mark):

Traditional programming requires loops:

```
# INEFFICIENT: Loop over each row
salaries = [50000, 45000, 60000, 52000]
bonuses = []
for salary in salaries:
    bonuses.append(salary * 0.1)
```

With DataFrames:

```
# EFFICIENT: Vectorized operation
df['Bonus'] = df['Salary'] * 0.1 # One-liner!
# Internally optimized in NumPy/C, much faster
```

Benefits:

- Dramatically faster for large datasets (100x+ speedup)
- Cleaner, more readable code
- Automatic memory optimization
- Parallelizable on multiple cores

Efficiency Aspect 2: Built-in Methods Eliminate Code (1 mark):

```
# Complex analysis in one line
df.groupby('Department')['Salary'].mean()
# Instead of: manual looping, accumulating, dividing

# Filtering in one line
young_high_earners = df[(df['Age'] < 30) & (df['Salary'] > 48000)]
# Instead of: nested loops and conditional checks

# Time series operations
df.set_index('Date').resample('M').sum()
# Instead of: manual date parsing, grouping, aggregation

# Missing data handling
df.fillna(df.mean())
# Instead of: iterating, identifying, replacing manually
```

Value for Data Visualization Programming:

- Prepare data in few lines
- Filter/aggregate before plotting
- Transform columns easily
- Combine multiple datasets efficiently
- Direct plotting integration: `df.plot()`

QUESTION 3: DATA VISUALIZATION GRAPHICS AND CHARACTERISTICS (25 marks)

OVERVIEW

Purpose: Identify which type of graph is suitable for which context and justify answers.

6 Graph Types to Analyze: Line graph, Bar chart graph, Histogram graph, Flow chart graph, Box plot graph, Bubble map graph.

Mark Breakdown: 4 marks × 6 graphs + 1 bonus = 25 marks

- Each graph: 4 marks = (1 mark) definition + (2 marks) best use contexts + (1 mark) justification

GRAPH 1: LINE GRAPH (4 marks)

Definition (1 mark):

Line graph connects data points with lines, showing values on y-axis against time/sequence on x-axis. Primary use: showing trends, changes, and progression over time.

Best Use Contexts (2 marks):

1. **Temporal Trends** (1 mark):

- Stock prices over months/years
- Website traffic by day
- Patient health metrics over treatment period
- Economic indicators (GDP, inflation) over decades

2. Comparison of Multiple Series Over Time (1 mark):

- Revenue vs. Expenses over quarters
- Different product sales trends on same chart
- Temperature highs vs. lows by month
- Multiple competitor performance tracking

Justification (1 mark):

- **Pattern Recognition:** Human eye instantly detects trends (rising, falling, stable)
 - **Seasonality:** Cyclical patterns become obvious
 - **Anomaly Detection:** Sudden spikes/drops catch attention
 - **Prediction:** Users can extrapolate future values
 - **Multiple Series:** Different colored lines enable comparison
 - **Efficient:** 2 dimensions (time × value) clearly displayed
 - **Intuitive:** Most familiar to general audiences
-

GRAPH 2: BAR CHART GRAPH (4 marks)

Definition (1 mark):

Bar chart displays categorical data as rectangular bars with heights proportional to values. Bars can be vertical (column chart) or horizontal. Enables direct magnitude comparison across categories.

Best Use Contexts (2 marks):

1. Categorical Comparison (1 mark):

- Sales by region/country
- Market share by competitor
- Preferred ice cream flavors (survey)
- Employee satisfaction by department
- Product revenue ranking

2. Part-to-Whole Relationships (1 mark) - *Stacked bar variant:*

- Company revenue breakdown by division
- Website traffic sources (organic, paid, direct, referral)
- Customer demographics (age groups)
- Budget allocation across departments

Justification (1 mark):

- **Precise Comparison:** Bar length directly proportional to value (accurate perception)
- **Simplicity:** Easy for any audience to understand
- **Flexibility:** Works with numerous categories without clutter

- **Composition Analysis:** Stacked variant shows both totals and breakdown
 - **Value Labels:** Can add exact numbers on bars
 - **Cultural Familiarity:** Most common graph type in business/media
-

GRAPH 3: HISTOGRAM GRAPH (4 marks)

Definition (1 mark):

Histogram shows distribution of continuous data using adjacent bars (bins), where height represents frequency. X-axis represents data ranges; Y-axis represents count/frequency.

Best Use Contexts (2 marks):

1. **Distribution Analysis** (1 mark):

- Customer age distribution
- Income levels in population
- Test score distribution in class
- Product defect rates
- Response times in call center

2. **Outlier and Spread Detection** (1 mark):

- Quality control: Identify out-of-specification items
- Medical screening: Detect abnormal values
- Statistical analysis: Check normality assumption
- Data cleaning: Identify data entry errors

Justification (1 mark):

- **Distribution Shape:** Instantly shows if data is normal, skewed, bimodal
 - **Central Tendency:** Where most data concentrates (mode)
 - **Spread:** How variable the data is
 - **Outliers:** Extreme values visible as isolated bars
 - **Data Quality:** Suspicious patterns reveal problems
 - **Statistical Foundation:** Basis for further analysis (normal distribution assumptions, etc.)
-

GRAPH 4: FLOW CHART GRAPH (4 marks)

Definition (1 mark):

Flow chart is a diagram showing processes, decision points, and data flow using connected boxes, diamonds, and arrows. Represents workflows, algorithms, or systems step-by-step.

Best Use Contexts (2 marks):

1. **Process Documentation** (1 mark):

- Software development workflows
- Customer service procedures
- Manufacturing processes

- Loan approval process
- Customer journey mapping

2. Decision Trees and Logic (1 mark):

- Diagnostic procedures (medical decision tree)
- Troubleshooting guides
- Business rules engine
- Algorithm flowchart
- Quality assurance testing procedures

Justification (1 mark):

- **Clarity:** Sequential steps instantly understood
- **Decision Visualization:** Shows branching logic (yes/no, pass/fail)
- **Communication:** Non-technical people understand processes
- **Optimization:** Identifies inefficiencies and bottlenecks
- **Training:** Helps new employees learn procedures
- **Documentation:** Serves as official reference material

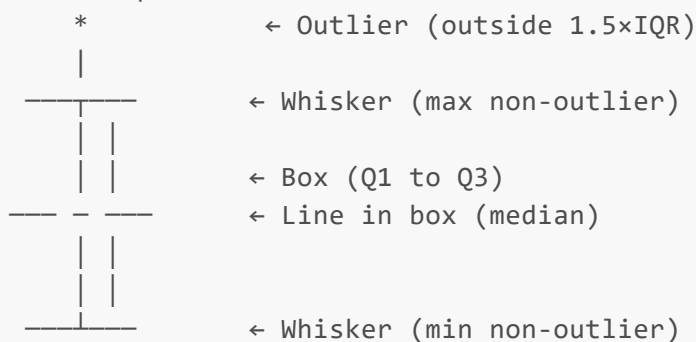
GRAPH 5: BOX PLOT GRAPH (4 marks)

Definition (1 mark):

Box plot (box-and-whisker) displays data distribution using five-number summary: minimum, Q1 (25th percentile), median (Q2), Q3 (75th percentile), maximum. Shows spread, skewness, and outliers.

Visual Structure:

Box Plot Components:



Best Use Contexts (2 marks):

1. Distribution Comparison Across Groups (1 mark):

- Salary distribution by department
- Test scores by school
- House prices by neighborhood
- Response times by service tier
- Production times by manufacturing plant

2. Outlier Identification (1 mark):

- Detect unusual measurements
- Identify exceptional cases for investigation
- Quality control: Flag defective units
- Fraud detection: Unusual transactions

Justification (1 mark):

- **Compact:** Shows entire distribution in small space
 - **Multiple Comparisons:** Side-by-side boxes compare groups
 - **Statistical Summary:** Displays key quantiles and spread
 - **Outlier Visibility:** Automatically identifies and marks anomalies
 - **Skewness Detection:** Asymmetric boxes show distribution bias
 - **Professional:** Standard in scientific/statistical reporting
-

GRAPH 6: BUBBLE MAP GRAPH (4 marks)

Definition (1 mark):

Bubble map is a geographic visualization overlaying bubbles (circles) on a map, where bubble size represents magnitude and color represents categorical variable. Combines geographic position with quantitative data.

Visual Example:

Map with Bubbles:

○ Large bubble (high value)

○ Medium ○ Large

○ Small (low value)

Legend: Size = Sales Volume, Color = Region/Category

Best Use Contexts (2 marks):

1. Geographic Distribution Analysis (1 mark):

- Sales by city/region
- Population density by country
- Store locations and performance
- Disease outbreaks by location
- Weather data by geographic point

2. Multi-Dimensional Geographic Analysis (1 mark):

- Sales (bubble size) by region (position) by product (color)
- Population (size) by country (location) by income level (color)

- Store performance (size) by location (position) by category (color)

Justification (1 mark):

- **Geographic Context:** Location provides immediate spatial understanding
- **Multiple Dimensions:** Size + color + position = 3+ data dimensions
- **Pattern Discovery:** Clusters and gaps visible spatially
- **Intuitive:** Most people understand maps
- **Business Value:** Sales/performance in context of physical locations
- **Engagement:** Interactive maps are compelling for presentations

SUMMARY TABLE: WHICH GRAPH FOR WHICH CONTEXT

Graph Type	Primary Use	Context	Example
Line	Trends over time	Sequential data	Stock price history
Bar	Category comparison	Discrete groups	Sales by region
Histogram	Distribution	Continuous data spread	Age distribution
Flow Chart	Process/logic	Sequential steps	Workflow diagram
Box Plot	Statistical summary	Group comparison + outliers	Salary by department
Bubble Map	Geographic analysis	Location + magnitude	Population by city

END OF THEORETICAL ANSWERS - 45 MARKS COMPLETE

Next section contains practical implementations of all 6 graph types in Python/Jupyter Notebook format.